R1.02 — Développement d'interfaces web : Architecture Client-Serveur et Protocole HTTP

Thème principal : Architecture Client-Serveur, protocole HTTP, méthodes GET/POST, codes de statut

Synthèse exécutive : Ce cours formalise le modèle d'interaction asymétrique Client-Serveur et analyse le protocole HTTP (HyperText Transfer Protocol), standard universel d'échange de ressources sur le Web. L'étudiant y découvre la nature 'stateless' (sans état) de HTTP et les mécanismes de contournement (cookies/sessions), le formatage textuel rigoureux des requêtes et des réponses, la distinction fonctionnelle et sécuritaire entre GET et POST, ainsi que la typologie des codes de statut HTTP (2xx, 3xx, 4xx, 5xx).

L'Architecture Client-Serveur et la Nature 'Stateless' de HTTP

L'architecture client-serveur régit l'intégralité des échanges sur le réseau Web. Contrairement aux réseaux Pair-à-Pair (P2P), le Web repose sur un modèle strictement asymétrique. Le client (navigateur web) émet une requête HTTP à destination d'un serveur web (processus applicatif distant comme Apache, Nginx ou IIS). Le serveur écoute sur un port réseau standardisé (port 80 pour HTTP, port 443 pour HTTPS), traite la demande et retourne une réponse HTTP. Le protocole HTTP est dit 'stateless' (sans état) : une fois la réponse transmise, la connexion est fermée et le serveur oublie instantanément le client. Cette absence de mémoire garantit une excellente scalabilité mais interdit le fonctionnement natif d'applications complexes comme les boutiques e-commerce. Pour pallier cela, on utilise des cookies : le serveur transmet un identifiant de session dans un en-tête 'Set-Cookie', que le navigateur enregistre et renvoie automatiquement dans les requêtes ultérieures.

- Asymétrie Client-Serveur : Rôles stricts entre le client actif qui demande et le serveur passif qui écoute.
- Protocole Stateless (Sans état) : Absence de mémoire des requêtes précédentes par le protocole HTTP.
- Mécanismes de Session : Utilisation de cookies et de sessions côté serveur pour simuler un état persistant.

Cas concret enseignant : *Le professeur compare HTTP à un guichetier amnésique : chaque fois que vous vous présentez devant lui, même si c'était il y a deux secondes, il vous traite comme un inconnu total. Pour qu'il se souvienne de vous, vous devez lui montrer votre badge (le cookie de session) à chaque phrase. Il ajoute qu'en chargeant la page d'accueil de l'IUT de Bayonne, le navigateur n'envoie pas une seule requête unique, mais des dizaines de requêtes HTTP simultanées pour récupérer séparément le HTML, les feuilles de style CSS, les scripts JS et les images.*

Structure des Messages HTTP et Comparatif GET vs POST

Un message HTTP transite sous forme textuelle brute. Une requête comporte : 1. La ligne de requête (verbe HTTP, URI, version) ; 2. Les en-têtes (headers) ; 3. Une ligne vide ; 4. Le corps (body). La réponse adopte la même structure mais commence par une ligne de statut. Le choix entre GET et POST est crucial : GET sert à lire des données sans modifier le serveur (idempotence), ses paramètres sont accolés à l'URL. POST est utilisé pour soumettre des données altérant le système, les paramètres transitent dans le corps (Body), permettant de transférer de grands volumes d'informations en toute discrétion.

- Message Textuel Normalisé : Requête et réponse structurées en texte clair avec une première ligne, des en-têtes (headers), une ligne vide et un corps (body).
- Méthode GET : Lecture et consultation de ressources, paramètres visibles dans l'URL, méthode sûre et idempotente.
- Méthode POST : Modification de l'état du serveur, paramètres encapsulés dans le corps de la requête.

Cas concret enseignant : *Le professeur met en garde contre l'utilisation absurde de GET pour transmettre un mot de passe : soumettre un mot de passe en GET l'affiche en clair dans l'URL, le sauvegarde dans l'historique du navigateur et l'enregistre en clair dans les fichiers de logs du serveur web. C'est la vulnérabilité parfaite au 'shoulder surfing' (regard par-dessus l'épaule).*

Décodage et Classification des Codes de Statut HTTP

Chaque réponse émise par le serveur inclut un code numérique à trois chiffres permettant au client d'interpréter l'issue de la demande : 2xx (Succès), 3xx (Redirection), 4xx (Erreur Client, ex: 404 Not Found), 5xx (Erreur Serveur, ex: 500 Internal Server Error).

- • Ligne de Statut de Réponse : Première ligne d'une réponse HTTP indiquant le résultat du traitement à l'aide d'un code à 3 chiffres.
- • Familles de Codes : 2xx (Succès), 3xx (Redirection), 4xx (Erreur Client), 5xx (Erreur Serveur).
- • Frontière de Responsabilité : Distinguer l'erreur imputable au client (4xx) du dysfonctionnement interne du serveur (5xx).

Cas concret enseignant : *Le professeur raconte une anecdote personnelle : à ses débuts, une erreur de syntaxe dans son script PHP (un simple point-virgule manquant) faisait planter la connexion à la base de données lors de la validation d'un formulaire. Le serveur renvoyait systématiquement une erreur 500 Internal Server Error. Les utilisateurs paniquaient en pensant que le site était piraté alors qu'il s'agissait simplement d'une faute de frappe dans son code backend.*

■ Questions & Interventions des Étudiants en Classe

Question étudiant : Monsieur, quelle est la différence concrète entre GET et POST pour envoyer un formulaire de connexion ?
Réponse du professeur : GET transmet les données dans l'URL (visible, limité en taille, non sécurisé pour les mots de passe), tandis que POST les envoie cachées dans le corps (body) de la requête, ce qui est indispensable pour la confidentialité et l'intégrité des données d'authentification.

■ Exercice & Démonstration d'Application

Énoncé : Analyser la requête HTTP brute suivante. Identifier la méthode utilisée, la ressource demandée, l'hôte cible, le rôle de l'agent utilisateur, et expliquer le comportement exact du serveur avec son code de statut si la ressource est inexistante : GET /images/logo.png HTTP/1.1 Host: iutbayonne.univ-pau.fr User-Agent: Mozilla/5.0

Résolution pas à pas :
1. Analyse de la ligne de requête ('GET /images/logo.png HTTP/1.1') : Méthode GET (récupération de ressource sans modification), URI '/images/logo.png', Version HTTP/1.1. 2. Analyse de l'en-tête 'Host' : Spécifie le nom de domaine du serveur destinataire, obligatoire en HTTP/1.1 pour le 'Virtual Hosting'. 3. Analyse de l'en-tête 'User-Agent' : Transmet la signature du logiciel client (navigateur). 4. Scénario d'absence de la ressource : Si le fichier n'existe pas, l'erreur provient d'une URL erronée formulée par le client, la responsabilité lui incombe.

Résultat : Méthode : GET | Ressource : /images/logo.png | Hôte cible : iutbayonne.univ-pau.fr | Rôle User-Agent : Identification du client | Code en cas d'absence : 404 Not Found (Erreur sous la responsabilité du client).

■ Auto-Évaluation Flash d'Examen

Q : Pourquoi qualifie-t-on le protocole HTTP de 'stateless' (sans état) et quelle est sa conséquence directe ?

R : Il ne conserve aucune mémoire des requêtes précédentes. Conséquence : chaque requête doit être indépendante et contenir toutes les informations d'authentification nécessaires (ex: cookies).

Q : Quelle est la différence fondamentale de responsabilité entre une erreur HTTP de la série 4xx et une erreur de la série 5xx ?

R : Les erreurs 4xx relèvent de la responsabilité du client (mauvaise URL, non authentifié), tandis que les erreurs 5xx indiquent un dysfonctionnement interne du serveur.

Q : Pourquoi la méthode HTTP GET ne doit-elle jamais être employée pour transmettre des données sensibles comme un mot de passe ?

R : Parce que les paramètres GET sont affichés en clair dans l'URL, enregistrés dans l'historique du navigateur et consignés dans les fichiers de logs du serveur.